

# ELI — Dag

ELI v2.1.81

August 2026

## Contents

|                                                                         |          |
|-------------------------------------------------------------------------|----------|
| <b>ELI DAG Engine — project-wide dependency scheduling</b>              | <b>1</b> |
| The engine (eli/core/dag.py)                                            | 1        |
| Consumer 1 — the agent bus runs on the DAG (eli/cognition/agent_bus.py) | 1        |
| Consumer 2 — the coding engine's subtask DAG (eli/coding/plan_graph.py) | 2        |
| Scope & honesty                                                         | 2        |
| Control summary                                                         | 2        |

## ELI DAG Engine — project-wide dependency scheduling

A single, generic directed-acyclic-graph engine (eli/core/dag.py) is now the shared scheduling primitive across ELI: the **agent bus** runs its agents on it, and the **coding engine** runs its subtasks on it. One DAG algorithm, two consumers — no per-subsystem reinvention.

### The engine (eli/core/dag.py)

Pure, deterministic, no I/O — fully unit-tested.

- DAG / DAGNode — nodes with depends\_on edges ( $B \text{ depends\_on } A \Rightarrow A \rightarrow B$ ).
- topological\_order() — Kahn's algorithm, deterministic (sorted ties); raises DAGCycleError on a cycle.
- topological\_layers() — groups nodes into **parallel layers**: layer  $k$  holds every node whose dependencies are all in earlier layers. This is what lets independent work run concurrently while dependent work waits.
- ancestors() / descendants() — transitive closure.
- critical\_path\_length() — longest dependency chain = number of sequential layers (a parallelism metric).
- build\_dag({node: [deps]}) — convenience builder that **drops deps outside the given node set**, so a global dependency map can be applied to any subset safely.

### Consumer 1 — the agent bus runs on the DAG (eli/cognition/agent\_bus.py)

Previously the bus was a single-round star fan-out (agents couldn't use each other's output). Now:

- \_AGENT\_DEPENDENCIES declares edges among agents. Shipped edge: knowledge\_graph  $\leftarrow$  memory.
- \_agent\_execution\_layers(active\_names) builds a DAG from that map  $\cap$  the selected agents and returns topological layers.
- AgentBus.\_run\_agents\_layered runs **layer by layer**: each layer's agents run in parallel (same per-agent hard timeout + failure handling as before), then every completed layer's results are passed to downstream agents via intent["\_upstream"]. The knowledge\_graph agent uses this to seed its lookup with what memory surfaced this turn.

- **Non-regressive:** when no edges apply to the selected set, the DAG collapses to a single layer = the legacy flat fan-out. Gated by `ELI_AGENT_DAG` (default on); any error falls back to flat dispatch (`_run_agents_flat`).

```
selected agents → _agent_execution_layers → [[memory, reflection, ...], [knowledge_graph], ...]
                                         | run parallel           | receives _upstream={memory:...}
```

Add an edge by editing `_AGENT_DEPENDENCIES`; cycles are rejected at build time.

## Consumer 2 – the coding engine’s subtask DAG (eli/coding/plan\_graph.py)

For decomposable coding tasks:

- `decompose_dag(task, generate)` asks the planner for a **graph** of subtasks (`{nodes:[{id, task, depends_on}]}`), builds a DAG, and validates it (cycle/ dangling-edge safe). A cohesive task yields a single node.
- `solve_dag(...)` walks the topological order, solving each node via the normal single-shot tree search and **feeding each node the code of its dependencies**, then `compose()` concatenates the node solutions (imports hoisted/deduped) into one module. `CodeAgent.solve` verifies the composed module against tests synthesized for the *original* task.
- Single-node tasks return `None` so the agent uses the plain single-shot path (no DAG overhead). Gated by `ELI_CODING_DAG` (default on).

## Scope & honesty

- **DAG-driven now:** agent-bus execution (topological layers + upstream) and coding subtask decomposition both run on eli/core/dag.
- **Still linear (not yet DAG):** the 12-stage orchestrator pipeline is a fixed chain; `execution_planner.ExecutionPlan.steps` is an ordered list. Converting those to DAGs is a possible next step but wasn't required for "agents on a DAG".
- **Composition is v1:** the coding DAG composes node code by ordered concatenation with import-deduping; integration quality scales with the model.
- Everything here is covered by tests/`test_dag.py` (engine, layered dispatch via fake agents, coding subtask DAG) — all deterministic.

## Control summary

| Knob                | Default                     | Effect                                                                  |
|---------------------|-----------------------------|-------------------------------------------------------------------------|
| ELI_AGENT_DAG       | on                          | agent-bus topological layered dispatch (off $\Rightarrow$ flat fan-out) |
| ELI_CODING_DAG      | on                          | coding subtask-DAG decomposition (off $\Rightarrow$ single-shot only)   |
| _AGENT_DEPENDENCIES | {knowledge_graph: {memory}} | declare agent edges                                                     |